

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 5

Name: S Svava Haasini

Roll No: 2520090170

Task 1: Directory Navigation and Previous Directory Handling

Question:

Implement directory navigation functionality to change directories, validate paths, update the current working directory, handle errors, maintain the previous directory, and test navigation.

Source Code:

```
haasini@SSH:~/OSSP_upload/Skill_Week5$ cat > T1.c <<'EOF'
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main(void)
{
    char current[PATH_MAX];
    char previous[PATH_MAX];
    char input[PATH_MAX];

    if (getcwd(current, sizeof(current)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    strcpy(previous, current);

    while (1)
    {
        if (getcwd(current, sizeof(current)) == NULL)
        {
            perror("getcwd");
            return 1;
        }

        printf("Current directory: %s\n", current);
        printf("Enter directory (or '-' for previous, 'exit' to stop): ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        /* Remove newline / carriage return */
        input[strcspn(input, "\r\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting directory navigation program.\n");
            break;
        }

        char old[PATH_MAX];
        EOF return 0;f("\n");y(previous, old);
    }
}
```

Output:

```
haasini@SSH:~/OSSP_upload/Skill_Week5$ gcc -Wall -Wextra T1.c -o T1
./T1
Current directory: /home/haasini/OSSP_upload/Skill_Week5
Enter directory (or '-' for previous, 'exit' to stop): ..

Current directory: /home/haasini/OSSP_upload
Enter directory (or '-' for previous, 'exit' to stop): -

Current directory: /home/haasini/OSSP_upload/Skill_Week5
Enter directory (or '-' for previous, 'exit' to stop): /notexist
cd: No such file or directory

Current directory: /home/haasini/OSSP_upload/Skill_Week5
Enter directory (or '-' for previous, 'exit' to stop): exit
Exiting directory navigation program.
haasini@SSH:~/OSSP_upload/Skill_Week5$
```

Observations:

1. `getcwd()` retrieves the current working directory and stores it in the current buffer, enabling the program to track the present location.
2. `chdir()` changes the process's working directory to the specified path, updating the internal directory state.
3. The program maintains a previous directory buffer that stores the last directory before navigation, enabling the '-' shortcut to return to the prior location.
4. Invalid or nonexistent directory paths are detected when `chdir()` fails, and an error message is displayed without terminating the program.
5. The program displays the current directory after each navigation attempt, providing visual confirmation of state changes.
6. The '-' input triggers navigation back to the previous directory by swapping the current and previous directory values.
7. The exit command terminates the navigation loop and the program, allowing graceful shutdown.
8. The program demonstrates proper error handling by continuing execution even after failed navigation attempts, maintaining shell functionality.

Task 2: Built-in Command Dispatch Table

Question:

Identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain shell state, and test dispatch logic.

Source Code:

```
mini-shell> ^C
haasini@SSH:~/OSSP_upload/Skill_Week5$ cat > T2.c <<'EOF'
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

void builtin_pwd(void)
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void builtin_hello(void)
{
    printf("Hello from built-in command!\n");
}

void builtin_help(void)
{
    printf("Available built-ins: pwd, hello, help, exit\n");
}

struct Builtin
{
    char *name;
    void (*function)(void);
};

int main(void)
{
    char command[50];

    struct Builtin builtins[] =
    {
        {"pwd", builtin_pwd},
        {"hello", builtin_hello},
        {"help", builtin_help}
    };

    int count = sizeof(builtins) / sizeof(builtins[0]);

    while (1)
    {
        printf("mini-shell> ");

EOF return 0;rintf("Invalid command: %s\n", command);0)
```

Output:

```

    for return 0, printf("Invalid command: %s\n", command); 0)
haasini@SSH:~/OSSP_upload/Skill_Week5$ gcc -Wall -Wextra T2.c -o T2
./T2
mini-shell> pwd
/home/haasini/OSSP_upload/Skill_Week5
mini-shell> hello
Hello from built-in command!
mini-shell> help
Available built-ins: pwd, hello, help, exit
mini-shell> abs
Invalid command: abs
mini-shell> abc
Invalid command: abc
mini-shell> exit
Exiting shell.

```

Observations:

1. Built-in commands (pwd, hello, help, exit) execute within the current process without creating child processes, maintaining shell state.
2. A dispatch table is implemented as a struct array containing command name strings paired with corresponding function pointers.
3. Function pointers in the Builtin struct enable dynamic invocation of the appropriate built-in command handler based on user input.
4. The pwd command calls getcwd() and displays the current working directory path.
5. The hello command prints a custom greeting message, demonstrating simple in-process command execution.
6. The help command lists all available built-in commands available to users.
7. Invalid commands (abc, abs) are detected by comparing against the dispatch table and an error message is displayed.
8. The exit command terminates the mini-shell loop, allowing controlled program termination.

Task 3: Current Directory, Exit Processing and Shell State

Question:

Retrieve the current directory, display the path, process exit requests, clean up resources, save state, and verify correct behavior.

Source Code:

```
Existing shell:
haasini@SSH:~/OSSP_upload/Skill_Week5$ cat > T3.c <<'EOF'
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main(void)
{
    char command[50];
    char cwd[PATH_MAX];

    printf("Mini shell started.\n");

    while (1)
    {
        printf("shell> ");

        if (scanf("%49s", command) != 1)
            break;

        if (strcmp(command, "pwd") == 0)
        {
            if (getcwd(cwd, sizeof(cwd)) != NULL)
                printf("Current directory: %s\n", cwd);
            else
                perror("getcwd");
        }
        else if (strcmp(command, "exit") == 0)
        {
            printf("Cleaning up resources...\n");
            printf("Saving shell state...\n");
            printf("Shell terminated successfully.\n");
            break;
        }
        else
        {
            printf("Unknown command: %s\n", command);
        }
    }

    return 0;
}
EOF
```

Output:

```
haasini@SSH:~/OSSP_upload/Skill_Week5$ gcc -Wall -Wextra T3.c -o T3
./T3
Mini shell started.
shell> pwd
Current directory: /home/haasini/OSSP_upload/Skill_Week5
shell> abc
Unknown command: abc
shell> exit
Cleaning up resources...
Saving shell state...
Shell terminated successfully.
```

Observations:

1. The mini shell starts by displaying 'Mini shell started' to indicate successful initialization.
2. `getcwd()` retrieves the current working directory and `pwd` displays its full path when the command is executed.
3. The shell enters an infinite loop accepting user commands via `scanf()` with the 'shell>' prompt.
4. Invalid commands (abc) are detected and an 'Unknown command' message is displayed without terminating the shell.
5. The exit command is processed by the else-if condition checking for string match with `strcmp()`.
6. Upon exit, cleanup messages are printed in sequence: 'Cleaning up resources...', 'Saving shell state...', and 'Shell terminated successfully.'
7. These cleanup messages demonstrate proper state management and resource deallocation before termination.
8. The break statement terminates the command loop, allowing the program to exit gracefully after cleanup.

Task 4: Environment Variable Export and Child Inheritance

Question:

Parse export syntax, update environment variables, validate names, handle existing variables, export variables to child processes, and test functionality.

Source Code:

```
haasini@SSH: ~/OSSP_upload × +
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    char input[200];

    printf("Enter export command (example: COURSE=OSSP): ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    char *equal = strchr(input, '=');

    if (equal == NULL)
    {
        printf("Invalid export syntax. Use NAME=value\n");
        return 1;
    }

    *equal = '\0';

    char *name = input;
    char *value = equal + 1;

    if (strlen(name) == 0)
    {
        printf("Variable name cannot be empty.\n");
        return 1;
    }

    if (setenv(name, value, 1) == -1)
    {
        perror("setenv");
        return 1;
    }

    printf("Environment variable updated.\n");
    printf("%s=%s\n", name, getenv(name));

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Inherited %s=%s\n", name, getenv(name));
        return 0;
    }

    wait(NULL);

    printf("\nParent confirms child completed.\n");

    return 0;
}
```

Output:

```
haasini@SSH: ~/OSSP_upload/Skill_Week5$ gcc main.c -o T4
haasini@SSH:~/OSSP_upload/Skill_Week5$ ./T4
Enter export command (example: COURSE=OSSP): COURSE=OSSP
Environment variable updated.
COURSE=OSSP

Child Process
Child PID: 1560
Inherited COURSE=OSSP

Parent confirms child completed.
haasini@SSH:~/OSSP_upload/Skill_Week5$ K
```

Observations:

1. The program prompts for export commands in the format 'NAME=value', with an example 'COURSE=OSSP' provided to users.
2. `strchr()` locates the '=' separator in the input string to parse the variable name and value components.
3. `setenv()` creates or updates the environment variable with the specified name and value in the parent process.
4. `getenv()` retrieves and displays the newly updated environment variable value to confirm successful setting.
5. `fork()` creates a child process that inherits a copy of the parent's environment, including the newly set variable.
6. The child process displays its own PID and the inherited `COURSE=OSSP` environment variable using `getenv()`.
7. `wait()` blocks the parent process until the child completes execution, ensuring proper synchronization and output order.
8. The parent confirms child completion with a message, demonstrating inter-process coordination and successful environment variable inheritance.